Work Plan: LSTM-Based Stock Market Performance Prediction

Author: opencode
Date: October 2, 2026

---

1. Problem Statement & Objectives

We need to build a model to predict stock market performance using Long Short-Term Memory (LSTM) networks. The work plan must be created before implementation begins.

Primary Objectives

1. Collect and preprocess historical stock market data suitable for time series forecasting
2. Design and implement an LSTM model to predict stock price movements/performance
3. Train, validate, and evaluate the model's predictive performance
4. Document the approach, methodology, and results

---

2. Data Collection

Goal: Acquire high-quality historical stock data for training and testing.

Data Requirements

- Historical price data (OHLC: Open, High, Low, Close) at daily or intraday resolution
- Sufficient time span (e.g., 3-5+ years) to capture market patterns and cycles
- Clean, adjusted prices (accounting for splits/dividends)

Potential Data Sources

1. **Yahoo Finance** (yfinance) - Free, easy API access, widely used
2. **Alpha Vantage** - Free tier available with rate limits
3. **Quandl/Nasdaq Data Link** - Historical datasets
4. **Polygon.io** - Real-time/historical with API
5. **Tiingo** - Financial data API

Action Items

- [ ] Select target stock(s) and time period for initial experimentation
- [ ] Choose appropriate data source and obtain API access if needed
- [ ] Download historical data and store in appropriate format (CSV/Parquet)

---

3. Data Preprocessing & Feature Engineering

Goal: Prepare time series data in a format suitable for LSTM training.

Preprocessing Steps

1. **Data cleaning**: Handle missing values, outliers, ensure chronological order
2. **Feature selection**: Start with Close price; consider additional features (Volume, technical indicators)
3. **Normalization/Scaling**: Apply MinMaxScaler or StandardScaler to scale values to [0,1] range (critical for LSTMs)
4. **Train-test split**: Split chronologically (e.g., 80% train, 20% test) - do not use random split for time series
5. **Sequence creation**: Create sliding windows of historical data to predict future values (e.g., use past 60 days to predict next day)

Feature Engineering (Optional but Recommended)

- Moving averages (MA5, MA20, MA50)
- Relative Strength Index (RSI)
- Moving Average Convergence Divergence (MACD)
- Volume-based indicators
- Lag features

Action Items

- [ ] Clean and validate data
- [ ] Implement scaling and inverse scaling functions
- [ ] Create time series sequences (X, y) for LSTM input
- [ ] Split data maintaining temporal order

---

4. Model Design (LSTM Architecture)

Goal: Design an effective LSTM architecture for stock price prediction.

Architecture Components

1. **Input layer**: Shape based on sequence length and number of features
2. **LSTM layers**: One or more stacked LSTM layers with appropriate units (e.g., 50, 100, 200)
3. **Dropout/Regularization**: Add dropout to prevent overfitting (e.g., 0.2-0.3)
4. **Dense layers**: Fully connected layers to output predictions
5. **Output layer**: Single neuron for regression (predicting price/value) or multiple for classification (up/down)

Model Type Decisions

- **Regression**: Predict exact future price (continuous)
- **Classification**: Predict direction (price up/down) - often easier to evaluate meaningfully
- **Multi-step**: Predict multiple future time steps

Hyperparameters to Consider

- Sequence length (lookback window): e.g., 30, 60, 90 days
- Number of LSTM units
- Batch size, epochs
- Learning rate, optimizer (Adam recommended)
- Loss function (MSE for regression, binary crossentropy for classification)

Action Items

- [ ] Define prediction task (regression vs classification)
- [ ] Design LSTM architecture
- [ ] Select hyperparameters and optimization strategy

---

5. Training & Validation

Goal: Train the model while preventing overfitting.

Training Strategy

1. **Validation split**: Use validation set from training data (e.g., last 10% of training or time-based split)
2. **Early stopping**: Monitor validation loss and stop when it stops improving
3. **Learning rate scheduling**: Adjust learning rate if validation loss plateaus
4. **Batch processing**: Choose appropriate batch size

Implementation Steps

- [ ] Build and compile the model
- [ ] Train with early stopping and callbacks
- [ ] Monitor training/validation loss curves
- [ ] Tune hyperparameters as needed (grid/random search)

---

6. Evaluation & Metrics

Goal: Assess model performance objectively.

Regression Metrics (if predicting prices)

- **Mean Squared Error (MSE)** - Penalizes large errors
- **Root Mean Squared Error (RMSE)** - Interpretable in price units
- **Mean Absolute Error (MAE)** - Average absolute difference
- **Mean Absolute Percentage Error (MAPE)** - Percentage error
- **R^2 Score** - Explained variance

Classification Metrics (if predicting direction)

- **Accuracy** - Overall correct predictions
- **Precision, Recall, F1-Score** - For directional predictions
- **Confusion Matrix** - Detailed breakdown
- **ROC-AUC** - Area under ROC curve

Important Considerations

- Evaluate on unseen test data (chronologically after training)
- Compare against baseline models (e.g., simple moving average, naive forecast)
- Consider financial metrics: profitability of a trading strategy based on predictions (beyond statistical metrics)

Action Items

- [ ] Evaluate model on test set using appropriate metrics
- [ ] Generate visualizations (predicted vs actual prices)
- [ ] Compare against baseline
- [ ] Analyze results and identify limitations

---

7. Tools, Libraries & Environment

Recommended Tech Stack - **Python 3.8+** - **TensorFlow/Keras** or **PyTorch** for LSTM implementation - **yfinance** for data acquisition - **pandas, numpy** for data manipulation - **scikit-learn** for preprocessing and metrics - **matplotlib, seaborn, plotly** for visualization - **jupyter** for exploratory analysis

Setup Requirements

- [ ] Set up Python environment (virtualenv/conda)
- [ ] Install required dependencies
- [ ] Configure API keys if using paid data sources

---

8. Implementation Timeline & Deliverables

Phase 1: Data & Exploration (1-2 hours)

- [ ] Data collection and initial exploration
- [ ] EDA and visualization

Phase 2: Preprocessing (1 hour)

- [ ] Feature engineering and sequence preparation
- [ ] Train-test split and scaling

Phase 3: Model Development (1-2 hours)

- [ ] LSTM model design and implementation
- [ ] Initial training

Phase 4: Training & Tuning (1-2 hours)

- [ ] Hyperparameter tuning and optimization
- [ ] Validation

Phase 5: Evaluation & Documentation (1-2 hours)

- [ ] Model evaluation and comparison
- [ ] Generate results, plots, and documentation
- [ ] Finalize deliverables

Deliverables

1. **Jupyter Notebook** with complete implementation (data loading → preprocessing → training → evaluation)
2. **Trained model** (saved in .h5 or saved_model format)
3. **Documentation** (README with setup instructions, methodology, results)
4. **Visualizations** (training curves, predictions vs actual, etc.)
5. **requirements.txt** with dependencies

---

9. Risks & Mitigations

Risk	Impact	Mitigation
Overfitting due to small dataset/noisy data	High	Use dropout, early stopping, validation split, regularization
Time series leakage	High	Ensure strict chronological splits; no future data in training
Non-stationarity of stock data	High	Consider differencing, returns instead of prices, or more sophisticated approaches
Limited predictive power (efficient market hypothesis)	Medium-High	Focus on short-term patterns, include relevant features, set realistic expectations
Data quality issues	Medium	Validate data, handle missing values properly

---

10. Next Steps

1. **Review this work plan** and confirm approach
2. **Proceed to implementation** starting with data collection
3. **Iterate** based on initial results - may need to adjust architecture, features, or timeframe

Note: This is a planning document. Implementation will follow the steps outlined above with appropriate adjustments based on findings during exploration.